

LayoutEvent Laboratory Manual

Android Layout Managers and UI Event Handling

Android Development Lab

May 2026

Contents

1	LayoutEvent — Laboratory Manual	3
1.1	Table of Contents	3
1.2	1. Introduction	4
1.2.1	1.1 Purpose	4
1.2.2	1.2 Learning Objectives	4
1.2.3	1.3 Key Concepts	4
1.3	2. Project Structure (Structural View)	5
1.4	3. Application Flow (Top to Bottom)	6
1.5	4. Configuration Files	7
1.5.1	4.1 AndroidManifest.xml	7
1.5.2	4.2 app/build.gradle.kts	7
1.5.3	4.3 settings.gradle.kts	9
1.5.4	4.4 gradle.properties (excerpt)	9
1.5.5	4.5 gradle/libs.versions.toml (dependency versions)	9
1.6	5. Resource Files (XML — values/)	9
1.6.1	5.1 res/values/strings.xml	9
1.6.2	5.2 res/values/colors.xml	10
1.6.3	5.3 res/values/themes.xml	10
1.6.4	5.4 res/values-night/themes.xml	10
1.6.5	5.5 res/values-v23/themes.xml	11
1.6.6	5.6 res/values/dimens.xml and qualifiers	11
1.6.7	5.7 res/navigation/nav_graph.xml	11
1.6.8	5.8 res/menu/menu_main.xml	12
1.7	6. Layout Files (XML — res/layout/)	12
1.7.1	6.1 activity_main.xml — Primary UI (used at runtime)	12
1.7.2	6.2 content_main.xml — Navigation host (alternate)	17
1.7.3	6.3 fragment_first.xml	17
1.7.4	6.4 fragment_second.xml	20
1.8	7. Java Source Files (Top-to-Bottom Flow)	21
1.8.1	7.1 MainActivity.java — Execution order in onCreate	21
1.8.2	7.2 FirstFragment.java — Fragment lifecycle path	24
1.8.3	7.3 SecondFragment.java	26
1.8.4	7.4 Event → UI response map (MainActivity)	27
1.9	8. Complete Lifecycle Flow (Summary)	27
1.9.1	8.1 Activity lifecycle (launcher path)	27
1.9.2	8.2 Combined XML + Java timeline	28

1.9.3 8.3 Fragment lifecycle (when Navigation is enabled)	28
1.109. File Relationship Diagram	28
1.1110. Lab Exercises for Students	29
1.11.1Exercise 1 — Trace the launch path	29
1.11.2Exercise 2 — LinearLayout weights	29
1.11.3Exercise 3 — New TableRow	30
1.11.4Exercise 4 — TextWatcher enhancement	30
1.11.5Exercise 5 — GridLayout IDs	30
1.11.6Exercise 6 — Use dimens qualifier	30
1.11.7Exercise 7 — Enable Navigation path	30
1.11.8Exercise 8 — Toolbar menu	30
1.11.9Exercise 9 — Custom color	30
1.11.10Exercise 10 — Viva preparation	30
1.1211. Prerequisites Checklist	30
1.1312. Troubleshooting	31
1.14Appendix A — Quick Reference: Layout Managers	32
1.15Appendix B — Distribution Notes for Instructors	32

Chapter 1

LayoutEvent — Laboratory Manual

Course topic: Android Layout Managers and UI Event Handling

Package: com.example.layoutevent

Application ID: com.example.layoutevent

Minimum SDK: 26 | **Target SDK:** 36

1.1 Table of Contents

In the PDF and Word (.docx) versions, a linked table of contents is inserted automatically after the title page.

1. Introduction
2. Project Structure (Structural View)
3. Application Flow (Top to Bottom)
4. Configuration Files
5. Resource Files (XML — values/)
6. Layout Files (XML — res/layout/)
7. Java Source Files (Top-to-Bottom Flow)
8. Complete Lifecycle Flow (Summary)
9. File Relationship Diagram

- 10. Lab Exercises for Students
 - 11. Prerequisites Checklist
 - 12. Troubleshooting
-

1.2 1. Introduction

1.2.1 1.1 Purpose

LayoutEvent is an educational Android application that demonstrates:

- **Layout managers:** LinearLayout, TableLayout, ConstraintLayout, GridLayout, and CoordinatorLayout
- **UI events:** click listeners, TextWatcher for live text updates, and checkbox state changes
- **Modern Android practices:** View Binding, Material Design components, edge-to-edge display, and optional Navigation Component with Fragments

When you launch the app, **MainActivity** loads **activity_main.xml**, which contains all layout examples on a single scrollable screen. Event handlers in `MainActivity.onCreate()` attach listeners to views defined in that layout.

The project also includes **FirstFragment** and **SecondFragment** (with **content_main.xml** and **nav_graph.xml**) as an alternate Navigation-based structure. That path is **not** used by the current launcher activity but is included for extended labs on fragments and navigation.

1.2.2 1.2 Learning Objectives

After completing this lab, you should be able to:

1. Identify when to use horizontal LinearLayout, TableLayout, ConstraintLayout, and GridLayout
2. Wire `OnClickListener`, `OnCheckedChangeListener`, and `TextWatcher` to views
3. Use **View Binding** instead of `findViewById()`
4. Trace execution from `AndroidManifest.xml` → Activity → Layout inflation → Event callbacks
5. Explain the Activity lifecycle methods involved at startup

1.2.3 1.3 Key Concepts

Concept	Role in this app
View Binding	Type-safe access to views (<code>ActivityMainBinding</code> , etc.)
Layout inflation	XML → View hierarchy via <code>inflate()</code>
Event listeners	Code runs when user interacts (click, type, check)

Concept	Role in this app
CoordinatorLayout	Hosts AppBar, scrollable content, and FAB with coordinated behavior
Toast / Snackbar	Short feedback messages

1.3 2. Project Structure (Structural View)

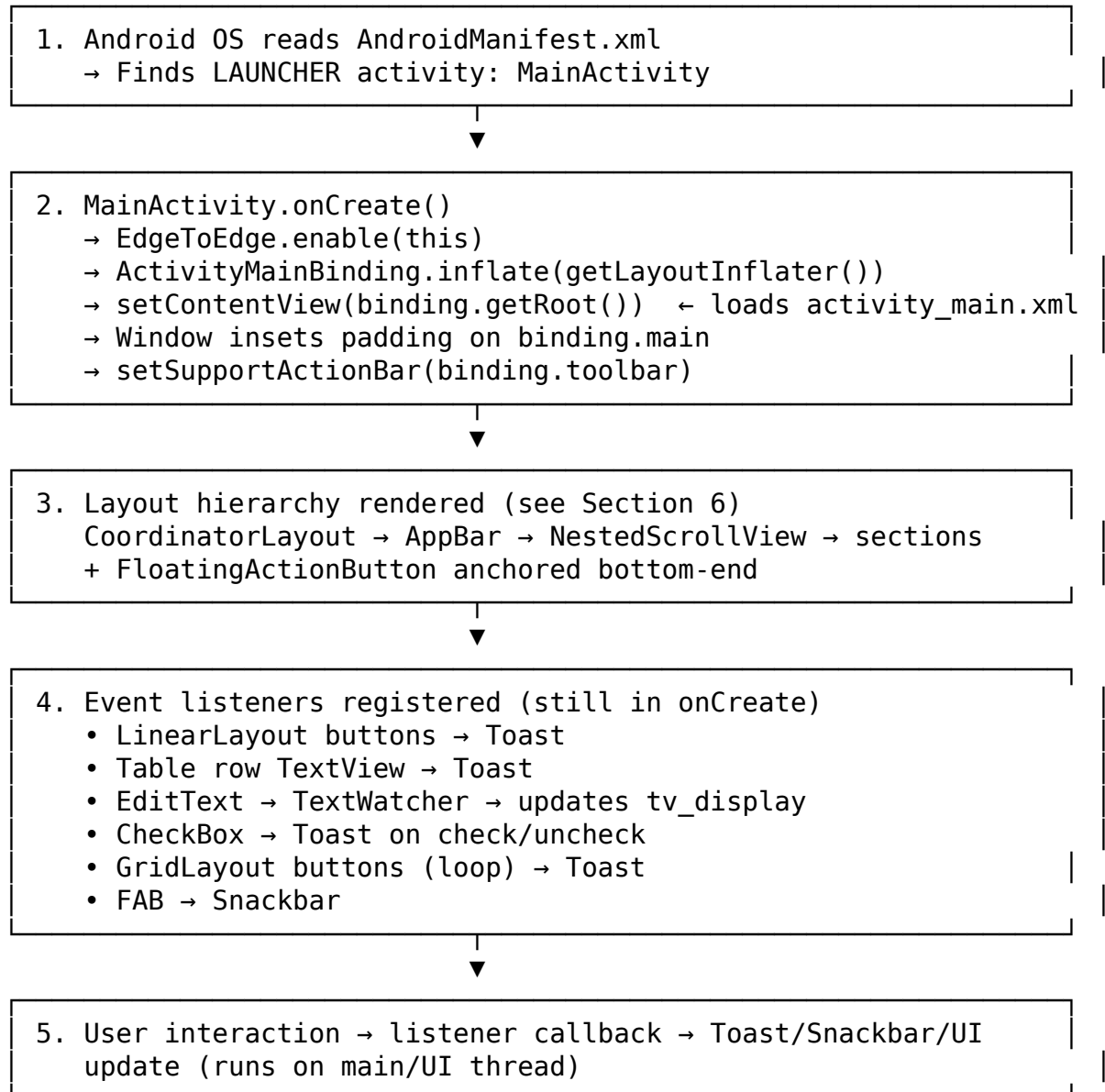
```

LayoutEvent/
├── app/
│   ├── build.gradle.kts          # App module: SDK, View Binding, dependencies
│   ├── proguard-rules.pro
│   └── src/
│       ├── main/
│       │   ├── AndroidManifest.xml
│       │   ├── java/com/example/layoutevent/
│       │   │   ├── MainActivity.java      ← Launcher (primary lab path)
│       │   │   ├── FirstFragment.java    ← Alternate: Navigation
│       │   │   └── SecondFragment.java    ← Alternate: Navigation
│       │   └── res/
│       │       ├── layout/
│       │       │   ├── activity_main.xml  ← Used by MainActivity
│       │       │   ├── content_main.xml   ← Nav host (alternate)
│       │       │   ├── fragment_first.xml
│       │       │   └── fragment_second.xml
│       │       ├── navigation/
│       │       │   └── nav_graph.xml
│       │       ├── menu/
│       │       │   └── menu_main.xml
│       │       └── values/                ← strings, colors, themes, dims
│       │           ├── values-night/
│       │           ├── values-v23/
│       │           ├── values-land/
│       │           ├── values-w600dp/
│       │           ├── values-w1240dp/
│       │           ├── drawable/
│       │           ├── mipmap-anydpi/
│       │           └── xml/                ← backup rules
│       ├── androidTest/                ← Instrumented tests
│       └── test/                       ← Unit tests
├── gradle/
│   ├── libs.versions.toml             # Version catalog
│   └── wrapper/
├── build.gradle.kts                   # Root Gradle
└── settings.gradle.kts

```

1.4 3. Application Flow (Top to Bottom)

Execution order when the user taps the app icon:



Alternate path (if you refactor `MainActivity` to use `content_main.xml`):

`MainActivity` → `content_main.xml` → `NavHostFragment` → `nav_graph.xml` → First-Fragment / SecondFragment → respective layout XML + Java listeners.

1.5 4. Configuration Files

1.5.1 4.1 AndroidManifest.xml

Declares the application package, theme, and the single launcher activity.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.LayoutEvent">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:theme="@style/Theme.LayoutEvent">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

Element	Meaning
android:theme	Applies Theme.LayoutEvent (Material 3, no action bar in theme—toolbar is in layout)
android:exported="true"	Required for launcher activity on modern Android
MAIN + LAUNCHER	Entry point when user opens the app

1.5.2 4.2 app/build.gradle.kts

```
plugins {
    alias(libs.plugins.android.application)
```



```

}

android {
    namespace = "com.example.layoutevent"
    compileSdk {
        version = release(36) {
            minorApiLevel = 1
        }
    }

    defaultConfig {
        applicationId = "com.example.layoutevent"
        minSdk = 26
        targetSdk = 36
        versionCode = 1
        versionName = "1.0"

        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
    }

    buildTypes {
        release {
            isMinifyEnabled = false
            proguardFiles(
                getDefaultProguardFile("proguard-android-optimize.txt"),
                "proguard-rules.pro"
            )
        }
    }

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_11
        targetCompatibility = JavaVersion.VERSION_11
    }

    buildFeatures {
        viewBinding = true
    }
}

dependencies {
    implementation(libs.activity.ktx)
    implementation(libs.appcompat)
    implementation(libs.constraintlayout)
    implementation(libs.material)
    implementation(libs.navigation.fragment)
    implementation(libs.navigation.ui)
    testImplementation(libs.junit)
    androidTestImplementation(libs.espresso.core)
    androidTestImplementation(libs.ext.junit)
}

```

```
}
```

Important: `viewBinding = true` generates `ActivityMainBinding` from `activity_main.xml` at build time.

1.5.3 4.3 settings.gradle.kts

```
rootProject.name = "LayoutEvent"  
include(":app")
```

1.5.4 4.4 gradle.properties (excerpt)

```
org.gradle.jvmargs=-Xmx2048m -Dfile.encoding=UTF-8
```

1.5.5 4.5 gradle/libs.versions.toml (dependency versions)

Key libraries: `AppCompat 1.7.1`, `Material 1.14.0`, `ConstraintLayout 2.2.1`, `Navigation 2.9.8`, `AGP 9.2.1`.

1.6 5. Resource Files (XML — values/)

Resources are compiled into R class fields (e.g. `R.string.app_name`).

1.6.1 5.1 res/values/strings.xml

All user-visible text is stored here in plain English (no placeholder Latin text).

```
<resources>  
    <string name="app_name">LayoutEvent</string>  
    <string name="action_settings">Settings</string>  
  
    <!-- Screen titles for Navigation Component -->  
    <string name="first_fragment_label">Layouts and Events</string>  
    <string name="second_fragment_label">Grid Layout Demo</string>  
    <string name="next">Go to Next Screen</string>  
    <string name="previous">Back to Previous Screen</string>  
  
    <!-- Short descriptions for the lab (optional use in layouts or help text) -->  
    <string name="app_description">LayoutEvent is a practice application for learning  
    <string name="section_linear_layout">LinearLayout places views in a single row  
    <string name="section_table_layout">TableLayout organizes content in rows and columns  
    <string name="section_constraint_layout">ConstraintLayout positions views using  
    <string name="section_grid_layout">GridLayout arranges buttons in a grid of rows and columns  
  
    <!-- Sample messages shown in Toasts (can be referenced from Java) -->
```

```

    <string name="toast_linear_left">Left button clicked (LinearLayout)</string>
    <string name="toast_linear_right">Right button clicked (LinearLayout)</string>
    <string name="toast_table_row">Table row item clicked</string>
    <string name="toast_checkbox_checked">Checkbox is checked</string>
    <string name="toast_checkbox_unchecked">Checkbox is unchecked</string>
    <string name="hint_type_here">Type here...</string>
    <string name="label_text_display">Text appears here as you type</string>
    <string name="label_checkbox">Check to show a message</string>
</resources>

```

Resource	Purpose
app_name	App title on launcher and manifest
first_fragment_label / second_fragment_label	Titles in Navigation graph
next / previous	Navigation button labels
app_description	Short overview of the lab app
section_*	Help text describing each layout section
toast_*	Ready-to-use messages for Toast.makeText(..., R.string.toast_..., ...)
hint_type_here / label_*	Suggested text for EditText, TextView, and CheckBox

1.6.2 5.2 res/values/colors.xml

```

<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFF</color>
</resources>

```

1.6.3 5.3 res/values/themes.xml

```

<resources xmlns:tools="http://schemas.android.com/tools">
    <!-- Base application theme. -->
    <style name="Base.Theme.LayoutEvent" parent="Theme.Material3.DayNight.NoActionBar"
        <!-- Customize your light theme here. -->
        <!-- <item name="colorPrimary">@color/my_light_primary</item> -->
    </style>

    <style name="Theme.LayoutEvent" parent="Base.Theme.LayoutEvent" />
</resources>

```

1.6.4 5.4 res/values-night/themes.xml

Dark theme variant (same parent; customize colorPrimary for night mode if needed).

1.6.5 5.5 res/values-v23/themes.xml

Edge-to-edge system bars (API 23+):

```
<resources xmlns:tools="http://schemas.android.com/tools">

    <style name="Theme.LayoutEvent" parent="Base.Theme.LayoutEvent">
        <!-- Transparent system bars for edge-to-edge. -->
        <item name="android:navigationBarColor">@android:color/transparent</item>
        <item name="android:statusBarColor">@android:color/transparent</item>
        <item name="android:windowLightStatusBar">?attr/isLightTheme</item>
    </style>
</resources>
```

1.6.6 5.6 res/values/dimens.xml and qualifiers

File	fab_margin
values/dimens.xml	16dp
values-land/dimens.xml	48dp
values-w600dp/dimens.xml	48dp
values-w1240dp/dimens.xml	200dp

Demonstrates **resource qualifiers** (orientation and smallest width). FAB in activity_main.xml uses fixed margins; fab_margin is available for exercises.

1.6.7 5.7 res/navigation/nav_graph.xml

```
<?xml version="1.0" encoding="utf-8"?>
<navigation xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/nav_graph"
    app:startDestination="@id/FirstFragment">

    <fragment
        android:id="@+id/FirstFragment"
        android:name="com.example.layoutevent.FirstFragment"
        android:label="@string/first_fragment_label"
        tools:layout="@layout/fragment_first">

        <action
            android:id="@+id/action_FirstFragment_to_SecondFragment"
            app:destination="@id/SecondFragment" />
    </fragment>
    <fragment
        android:id="@+id/SecondFragment"
        android:name="com.example.layoutevent.SecondFragment"
```

```

        android:label="@string/second_fragment_label"
        tools:layout="@layout/fragment_second">

        <action
            android:id="@+id/action_SecondFragment_to_FirstFragment"
            app:destination="@id/FirstFragment" />
    </fragment>
</navigation>

```

1.6.8 5.8 res/menu/menu_main.xml

```

<menu xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    tools:context="com.example.layoutevent.MainActivity">
    <item
        android:id="@+id/action_settings"
        android:orderInCategory="100"
        android:title="@string/action_settings"
        app:showAsAction="never" />
</menu>

```

Note: MainActivity sets the toolbar but does not inflate this menu in the current code—optional extension.

1.7 6. Layout Files (XML — res/layout/)

Layouts are inflated **top to bottom** as written in XML (parent first, then children in document order).

1.7.1 6.1 activity_main.xml — Primary UI (used at runtime)

Root: CoordinatorLayout (@+id/main)

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.coordinatorlayout.widget.CoordinatorLayout xmlns:android="http://schemas
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fitsSystemWindows="true"
    tools:context=".MainActivity">

    <com.google.android.material.appbar.AppBarLayout
        android:layout_width="match_parent"

```

```

        android:layout_height="wrap_content"
        android:fitsSystemWindows="true">

        <com.google.android.material.appbar.MaterialToolbar
            android:id="@+id/toolbar"
            android:layout_width="match_parent"
            android:layout_height="?attr/actionBarSize" />

    </com.google.android.material.appbar.AppBarLayout>

    <androidx.core.widget.NestedScrollView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        app:layout_behavior="@string/appbar_scrolling_view_behavior">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="16dp">

            <!-- 1. LinearLayout Example -->
            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="1. LinearLayout (Horizontal)"
                android:textSize="18sp"
                android:textStyle="bold" />

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:orientation="horizontal"
                android:layout_marginTop="8dp">

                <Button
                    android:id="@+id/btn_linear_left"
                    android:layout_width="0dp"
                    android:layout_height="wrap_content"
                    android:layout_weight="1"
                    android:text="Left Button" />

                <Button
                    android:id="@+id/btn_linear_right"
                    android:layout_width="0dp"
                    android:layout_height="wrap_content"
                    android:layout_weight="1"
                    android:text="Right Button" />

```

```

</LinearLayout>

<View
    android:layout_width="match_parent"
    android:layout_height="1dp"
    android:background="@android:color/darker_gray"
    android:layout_marginVertical="16dp" />

<!-- 2. TableLayout Example -->
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="2. TableLayout"
    android:textSize="18sp"
    android:textStyle="bold" />

<TableLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:stretchColumns="1"
    android:layout_marginTop="8dp">

    <TableRow>
        <TextView android:padding="8dp" android:text="ID" android:textStyle="bold" />
        <TextView android:padding="8dp" android:text="Name" android:textStyle="bold" />
    </TableRow>

    <TableRow>
        <TextView android:padding="8dp" android:text="001" />
        <TextView
            android:id="@+id/tv_table_item"
            android:padding="8dp"
            android:text="Clickable Row Item"
            android:textColor="@color/design_default_color_primary" />
    </TableRow>
</TableLayout>

<View
    android:layout_width="match_parent"
    android:layout_height="1dp"
    android:background="@android:color/darker_gray"
    android:layout_marginVertical="16dp" />

<!-- 3. ConstraintLayout Example -->
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="3. ConstraintLayout & Events"

```

```

        android:textSize="18sp"
        android:textStyle="bold" />

<androidx.constraintlayout.widget.ConstraintLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp">

    <EditText
        android:id="@+id/et_input"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:hint="Type here..."
        android:inputType="text"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <TextView
        android:id="@+id/tv_display"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:text="Text reflects here"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/et_input" />

    <CheckBox
        android:id="@+id/cb_listener"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:text="Check for Toast"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/tv_display" />

</androidx.constraintlayout.widget.ConstraintLayout>

<View
    android:layout_width="match_parent"
    android:layout_height="1dp"
    android:background="@android:color/darker_gray"
    android:layout_marginVertical="16dp" />

<!-- 4. GridLayout Example -->
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"

```



```

        android:text="4. GridLayout"
        android:textSize="18sp"
        android:textStyle="bold" />

<GridLayout
    android:id="@+id/grid_example"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:columnCount="2"
    android:layout_marginTop="8dp">

    <Button android:text="Grid 1" android:layout_columnWeight="1"/>
    <Button android:text="Grid 2" android:layout_columnWeight="1"/>
    <Button android:text="Grid 3" android:layout_columnWeight="1"/>
    <Button android:text="Grid 4" android:layout_columnWeight="1"/>
</GridLayout>

<!-- Spacer for bottom padding -->
<View
    android:layout_width="match_parent"
    android:layout_height="80dp" />

</LinearLayout>
</androidx.core.widget.NestedScrollView>

<com.google.android.material.floatingactionbutton.FloatingActionButton
    android:id="@+id/fab"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="bottom|end"
    android:layout_marginEnd="16dp"
    android:layout_marginBottom="16dp"
    app:srcCompat="@android:drawable/ic_dialog_info" />

</androidx.coordinatorlayout.widget.CoordinatorLayout>

```

View ID → Java binding mapping (generated names):

XML @+id	Binding field
main	binding.main
toolbar	binding.toolbar
btn_linear_left	binding.btnLinearLeft
btn_linear_right	binding.btnLinearRight
tv_table_item	binding.tvTableItem
et_input	binding.etInput
tv_display	binding.tvDisplay
cb_listener	binding.cbListener
grid_example	binding.gridExample

XML @+id	Binding field
fab	binding.fab

1.7.2 6.2 content_main.xml — Navigation host (alternate)

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res-auto"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <androidx.fragment.app.FragmentContainerView
        android:id="@+id/nav_host_fragment_content_main"
        android:name="androidx.navigation.fragment.NavHostFragment"
        android:layout_width="0dp"
        android:layout_height="0dp"
        app:defaultNavHost="true"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:navGraph="@navigation/nav_graph" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

1.7.3 6.3 fragment_first.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.core.widget.NestedScrollView xmlns:android="http://schemas.android.com/apk/res-auto"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".FirstFragment">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="LinearLayout Example (Horizontal)"
            android:textStyle="bold" />

    </LinearLayout>
</androidx.core.widget.NestedScrollView>
```

```

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:layout_marginTop="8dp">

    <Button
        android:id="@+id/btn_linear1"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="Button 1" />

    <Button
        android:id="@+id/btn_linear2"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="Button 2" />
</LinearLayout>

<View
    android:layout_width="match_parent"
    android:layout_height="1dp"
    android:background="@android:color/darker_gray"
    android:layout_marginVertical="16dp" />

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="TableLayout Example"
    android:textStyle="bold" />

<TableLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:stretchColumns="1">

    <TableRow>
        <TextView
            android:padding="8dp"
            android:text="Row 1, Col 1" />
        <TextView
            android:id="@+id/tv_table_row1"
            android:padding="8dp"
            android:text="Click me (Row 1, Col 2)" />
    </TableRow>

```

```

<TableRow>
    <TextView
        android:padding="8dp"
        android:text="Row 2, Col 1" />
    <Button
        android:id="@+id/btn_table_row2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Row 2 Button" />
</TableRow>
</TableLayout>

<View
    android:layout_width="match_parent"
    android:layout_height="1dp"
    android:background="@android:color/darker_gray"
    android:layout_marginVertical="16dp" />

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="ConstraintLayout Example (with Event Listeners)"
    android:textStyle="bold" />

<androidx.constraintlayout.widget.ConstraintLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp">

    <EditText
        android:id="@+id/et_input"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:hint="Type something..."
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <TextView
        android:id="@+id/tv_display"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:text="Your text will appear here"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/et_input" />

<CheckBox

```

```

        android:id="@+id/cb_toggle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:text="Show Toast on check"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/tv_display" />

        <Button
            android:id="@+id/button_first"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="16dp"
            android:text="Go to Second Fragment"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toBottomOf="@id/cb_toggle" />

    </androidx.constraintlayout.widget.ConstraintLayout>

</LinearLayout>
</androidx.core.widget.NestedScrollView>

```

1.7.4 6.4 fragment_second.xml

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.core.widget.NestedScrollView xmlns:android="http://schemas.android.com/apk/res-auto"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".SecondFragment">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="GridLayout Example"
            android:textStyle="bold"
            android:layout_marginBottom="8dp" />

        <GridLayout
            android:id="@+id/grid_layout"

```

```

        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:columnCount="3"
        android:rowCount="3">

        <Button android:text="1" android:layout_columnWeight="1"/>
        <Button android:text="2" android:layout_columnWeight="1"/>
        <Button android:text="3" android:layout_columnWeight="1"/>
        <Button android:text="4" android:layout_columnWeight="1"/>
        <Button android:text="5" android:layout_columnWeight="1"/>
        <Button android:text="6" android:layout_columnWeight="1"/>
        <Button android:text="7" android:layout_columnWeight="1"/>
        <Button android:text="8" android:layout_columnWeight="1"/>
        <Button android:text="9" android:layout_columnWeight="1"/>

    </GridLayout>

    <Button
        android:id="@+id/button_second"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:layout_marginTop="32dp"
        android:text="@string/previous" />

</LinearLayout>
</androidx.core.widget.NestedScrollView>

```

1.8 7. Java Source Files (Top-to-Bottom Flow)

Java executes **after** layouts exist. Order within each class matters.

1.8.1 7.1 MainActivity.java — Execution order in onCreate

```

package com.example.layoutevent;

import android.os.Bundle;
import android.text.Editable;
import android.text.TextWatcher;
import android.view.View;
import android.widget.Button;
import android.widget.Toast;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;

```

```

import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

import com.example.layoutevent.databinding.ActivityMainBinding;
import com.google.android.material.snackbar.Snackbar;

public class MainActivity extends AppCompatActivity {

    private ActivityMainBinding binding;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);

        binding = ActivityMainBinding.inflate(getLayoutInflater());
        setContentView(binding.getRoot());

        ViewCompat.setOnApplyWindowInsetsListener(binding.main, (v, insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });

        setSupportActionBar(binding.toolbar);

        // --- Event Listeners for Layout Managers ---

        // 1. LinearLayout Button Listeners
        binding.btnLinearLeft.setOnClickListener(v ->
            Toast.makeText(this, "Left Button (LinearLayout) clicked", Toast.LENGTH_SHORT)
                .show());

        binding.btnLinearRight.setOnClickListener(v ->
            Toast.makeText(this, "Right Button (LinearLayout) clicked", Toast.LENGTH_SHORT)
                .show());

        // 2. TableLayout Item Listener
        binding.tvTableItem.setOnClickListener(v ->
            Toast.makeText(this, "Table row item clicked", Toast.LENGTH_SHORT).show());

        // 3. ConstraintLayout: EditText & CheckBox Listeners
        binding.etInput.addTextChangedListener(new TextWatcher() {
            @Override
            public void beforeTextChanged(CharSequence s, int start, int count, int end) {}
        });
    }
}

```

```

        @Override
        public void onTextChanged(CharSequence s, int start, int before, int count) {
            binding.tvDisplay.setText("Typed: " + s.toString());
        }

        @Override
        public void afterTextChanged(Editable s) {}
    });

    binding.cbListener.setOnCheckedChangeListener((buttonView, isChecked) -> {
        String state = isChecked ? "Checked" : "Unchecked";
        Toast.makeText(this, "Checkbox is " + state, Toast.LENGTH_SHORT).show();
    });

    // 4. GridLayout: Iterative listener assignment
    for (int i = 0; i < binding.gridExample.getChildCount(); i++) {
        View child = binding.gridExample.getChildAt(i);
        if (child instanceof Button) {
            Button btn = (Button) child;
            btn.setOnClickListener(v -> {
                Toast.makeText(this, btn.getText() + " clicked", Toast.LENGTH_SHORT).show();
            });
        }
    }

    // 5. FAB Listener (CoordinatorLayout)
    binding.fab.setOnClickListener(view -> {
        Snackbar.make(view, "Floating Action Button Clicked", Snackbar.LENGTH_LONG)
            .setAction("Undo", null).show();
    });
}
}

```

Step-by-step (matches runtime):

Step	Code	Effect
1	super.onCreate()	Activity lifecycle setup
2	EdgeToEdge.enable(this)	Draw behind status/navigation bars
3	ActivityMainBinding.inflate(activity_main.xml → views)	Parse activity_main.xml → views
4	setContentView(binding.getRoot())	Attach RootView to window
5	Window insets listener	Pad main for system bars
6	setSupportActionBar(toolbar)	Material toolbar as action bar
7-11	Listeners	Ready for user input

1.8.2 7.2 FirstFragment.java — Fragment lifecycle path

```
package com.example.layoutevent;

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;

import androidx.annotation.NonNull;
import androidx.fragment.app.Fragment;
import androidx.navigation.fragment.NavHostFragment;

import android.textEditable;
import android.text.TextWatcher;
import android.widget.Toast;

import com.example.layoutevent.databinding.FragmentFirstBinding;

public class FirstFragment extends Fragment {

    private FragmentFirstBinding binding;

    @Override
    public View onCreateView(
        @NonNull LayoutInflater inflater, ViewGroup container,
        Bundle savedInstanceState
    ) {

        binding = FragmentFirstBinding.inflate(inflater, container, false);
        return binding.getRoot();

    }

    public void onViewCreated(@NonNull View view, Bundle savedInstanceState) {
        super.onViewCreated(view, savedInstanceState);

        // Linear Layout Buttons Event Listeners
        binding.btnLinear1.setOnClickListener(v ->
            Toast.makeText(getContext(), "Linear Button 1 clicked", Toast.LENGTH_SHORT)
        );

        binding.btnLinear2.setOnClickListener(v ->
            Toast.makeText(getContext(), "Linear Button 2 clicked", Toast.LENGTH_SHORT)
        );

        // Table Layout View Event Listener
        binding.tvTableRow1.setOnClickListener(v ->
```

```

        Toast.makeText(getContext(), "Table Text clicked", Toast.LENGTH_SHORT)
    );

    binding.btnTableRow2.setOnClickListener(v ->
        Toast.makeText(getContext(), "Table Button clicked", Toast.LENGTH_SHORT)
    );

    // EditText Event Listener (TextWatcher)
    binding.etInput.addTextChangedListener(new TextWatcher() {
        @Override
        public void beforeTextChanged(CharSequence s, int start, int count, int
            remaining) {}

        @Override
        public void onTextChanged(CharSequence s, int start, int before, int co
            unt) {
            binding.tvDisplay.setText(s.toString().isEmpty() ? "Your text will
            be displayed here" : s.toString());
        }

        @Override
        public void afterTextChanged(Editable s) {}
    });

    // CheckBox Event Listener
    binding.cbToggle.setOnCheckedChangeListener((buttonView, isChecked) -> {
        if (isChecked) {
            Toast.makeText(getContext(), "Checkbox checked!", Toast.LENGTH_SHOR
            T);
        } else {
            Toast.makeText(getContext(), "Checkbox unchecked!", Toast.LENGTH_SH
            ORT);
        }
    });

    binding.buttonFirst.setOnClickListener(v ->
        NavHostFragment.findNavController(FirstFragment.this)
            .navigate(R.id.action_FirstFragment_to_SecondFragment)
    );
}

@Override
public void onDestroyView() {
    super.onDestroyView();
    binding = null;
}
}

```

Fragment order: onCreateView (inflate XML) → onViewCreated (listeners) → user events → onDestroyView (clear binding).

1.8.3 7.3 SecondFragment.java

```
package com.example.layoutevent;

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;

import androidx.annotation.NonNull;
import androidx.fragment.app.Fragment;
import androidx.navigation.fragment.NavHostFragment;

import android.widget.Button;
import android.widget.Toast;

import com.example.layoutevent.databinding.FragmentSecondBinding;

public class SecondFragment extends Fragment {

    private FragmentSecondBinding binding;

    @Override
    public View onCreateView(
        @NonNull LayoutInflater inflater, ViewGroup container,
        Bundle savedInstanceState
    ) {

        binding = FragmentSecondBinding.inflate(inflater, container, false);
        return binding.getRoot();

    }

    public void onViewCreated(@NonNull View view, Bundle savedInstanceState) {
        super.onViewCreated(view, savedInstanceState);

        // Set click listeners for all buttons in the GridLayout
        for (int i = 0; i < binding.gridLayout.getChildCount(); i++) {
            View child = binding.gridLayout.getChildAt(i);
            if (child instanceof Button) {
                Button btn = (Button) child;
                btn.setOnClickListener(v ->
                    Toast.makeText(getContext(), "Grid Button " + btn.getText() + "
                );
            }
        }

        binding.buttonSecond.setOnClickListener(v ->
```

```

        NavHostFragment.findNavController(SecondFragment.this)
            .navigate(R.id.action_SecondFragment_to_FirstFragment)
    );
}

@Override
public void onDestroyView() {
    super.onDestroyView();
    binding = null;
}
}

```

1.8.4 7.4 Event → UI response map (MainActivity)

User action	Listener type	Result
Tap Left/Right button	OnClickListener	Toast with button name
Tap table cell	OnClickListener	Toast "Table row item clicked"
Type in EditText	TextWatcher.onTextChanged	tv_display shows Typed:
		...
Toggle checkbox	OnCheckedChangeListener	Toast Checked/Unchecked
Tap grid button	OnClickListener (loop)	Toast with button label
Tap FAB	OnClickListener	Snackbar with optional Undo

1.9 8. Complete Lifecycle Flow (Summary)

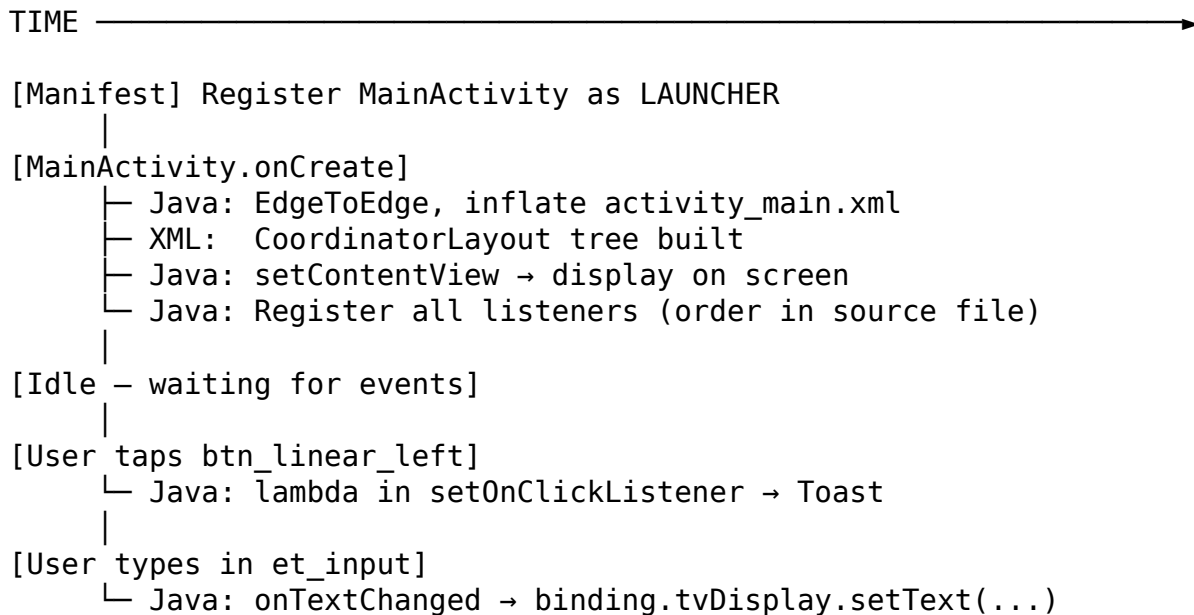
1.9.1 8.1 Activity lifecycle (launcher path)

```

onCreate()    → Inflate layout, register listeners
onStart()     → Activity visible (framework)
onResume()    → Interactive; user can tap views
    [USER INTERACTION – listeners fire]
onPause()     → Partially obscured
onStop()      → Not visible
onDestroy()   → Activity destroyed

```

1.9.2 8.2 Combined XML + Java timeline



1.9.3 8.3 Fragment lifecycle (when Navigation is enabled)

```
FirstFragment.onCreateView      → fragment_first.xml
FirstFragment.onViewCreated     → attach listeners
[Navigate to SecondFragment]
FirstFragment.onDestroyView
SecondFragment.onCreateView     → fragment_second.xml
SecondFragment.onViewCreated    → grid listeners + back navigation
```

1.10 9. File Relationship Diagram

flowchart TB

```
    subgraph config [Configuration]
        M[AndroidManifest.xml]
        G[app/build.gradle.kts]
        T[values/themes.xml]
    end
```

```
    subgraph resources [Resources]
        S[strings.xml]
        C[colors.xml]
        N[nav_graph.xml]
    end
```

```
    subgraph layouts [Layouts]
        AM[activity_main.xml]
    end
```

```

    CM[content_main.xml]
    FF[fragment_first.xml]
    FS[fragment_second.xml]
end

subgraph java [Java]
    MA[MainActivity.java]
    F1[FirstFragment.java]
    F2[SecondFragment.java]
end

M -->|launches| MA
M -->|theme| T
M -->|label| S
G -->|viewBinding| MA
MA -->|inflate| AM
AM -->|ids| MA
CM -->|navGraph| N
N --> F1
N --> F2
F1 -->|inflate| FF
F2 -->|inflate| FS

```

Primary runtime chain (bold):

AndroidManifest → MainActivity → activity_main.xml → ActivityMainBinding → Event listeners

Optional chain:

content_main.xml → NavHostFragment → nav_graph.xml → FirstFragment / SecondFragment → **fragment_*.xml**

1.11 10. Lab Exercises for Students

1.11.1 Exercise 1 — Trace the launch path

Draw a sequence diagram from app icon tap to first visible frame. Label manifest, onCreate, inflation, and setContentView.

1.11.2 Exercise 2 — LinearLayout weights

Change layout_weight on the two horizontal buttons so one occupies 75% width and the other 25%. Rebuild and verify on emulator.

1.11.3 Exercise 3 — New TableRow

Add a third row to `TableLayout` in `activity_main.xml` with a `Button`. Register a click listener in `MainActivity` that shows a `Snackbar` instead of `Toast`.

1.11.4 Exercise 4 — TextWatcher enhancement

In `MainActivity`, display character count in `tv_display` (e.g. Typed: hello (5 chars)).

1.11.5 Exercise 5 — GridLayout IDs

Give each grid `Button` an `@+id` in XML and set listeners individually (no loop). Compare with the loop approach.

1.11.6 Exercise 6 — Use dimens qualifier

Replace FAB hard-coded 16dp margins with `@dimen/fab_margin`. Test in portrait vs landscape.

1.11.7 Exercise 7 — Enable Navigation path

Refactor `MainActivity` to use `content_main.xml` (include toolbar + fragment host). Verify `FirstFragment` → `SecondFragment` navigation works. Document differences from single-activity layout.

1.11.8 Exercise 8 — Toolbar menu

Inflate `menu_main.xml` in `MainActivity` and show a `Toast` when `Settings` is selected.

1.11.9 Exercise 9 — Custom color

Add `color/primary_accent` in `colors.xml`, apply to clickable table text in `activity_main.xml`.

1.11.10 Exercise 10 — Viva preparation

Explain why `binding = null` in `onDestroyView()` for fragments but not required the same way in `Activity` for this lab.

1.12 11. Prerequisites Checklist

Before starting the lab, ensure:

- ☐ **Android Studio** installed (version compatible with AGP 9.2.x)

- ☐ **JDK 11+** configured (Project uses Java 11)
- ☐ **Android SDK** API 26–36 installed (SDK Manager)
- ☐ Emulator or physical device with **API ≥ 26**
- ☐ Basic **Java** (classes, interfaces, lambdas)
- ☐ Familiarity with **XML** syntax
- ☐ Understanding of **Activity** as a single screen controller
- ☐ Optional: prior exposure to **Toast**, **Logcat**, and **Gradle sync**

Setup steps:

1. Open project folder LayoutEvent in Android Studio
2. **File → Sync Project with Gradle Files**
3. Select run configuration **app**
4. Run on emulator/device
5. Open **Logcat** filter by package `com.example.layoutevent` for debugging

1.13 12. Troubleshooting

Problem	Likely cause	Solution
Cannot resolve symbol Activity-MainBinding	View Binding off or no rebuild	Enable viewBinding in app/build.gradle.kts; Build → Rebuild Project
Gradle sync failed	SDK/AGP mismatch	Install required SDK; update Android Studio
App crashes on launch	Inflation error in XML	Check Logcat stack trace; validate @id references in ConstraintLayout
Toast does not appear	Listener not registered	Confirm code is in onCreate after setContentView; correct binding field name
tv_display does not update	TextWatcher not attached	Verify et_input id matches binding; check onChanged body
Grid buttons no response	Loop before children added	Listeners must run after inflation; grid_example must contain Button children

Problem	Likely cause	Solution
FAB hidden behind nav bar	Insets	Edge-to-edge padding is on main; adjust FAB margin or insets
Fragment navigation not working	content_main not used	Current MainActivity uses activity_main only; complete Exercise 7 to wire Navigation
R.id errors after rename	Stale build	Build → Clean Project , then Rebuild
Emulator too slow	Low RAM allocation	Increase AVD RAM; use x86_64 image with hardware acceleration

Logcat tip: Filter by tag `AndroidRuntime` for crash stack traces. Search for `Caused by:` line.

View Binding naming: `btn_linear_left` → `btnLinearLeft` (snake_case XML to camelCase Java).

1.14 Appendix A — Quick Reference: Layout Managers

Layout	Child arrangement	Example in app
<code>LinearLayout</code>	Single row or column	Two equal-width buttons
<code>TableLayout</code>	Rows and columns	ID / Name table
<code>ConstraintLayout</code>	Constraints to parent/siblings	<code>EditText</code> , <code>TextView</code> , <code>CheckBox</code> chain
<code>GridLayout</code>	2D grid	2×2 or 3×3 buttons
<code>CoordinatorLayout</code>	Coordinated scrolling + FAB	Toolbar + <code>NestedScrollView</code> + FAB

1.15 Appendix B — Distribution Notes for Instructors

- Export this file as PDF via Android Studio / Markdown preview / Pandoc for hand-outs.
- Recommended lab duration: **2-3 hours** (Exercises 1-6 core; 7-10 advanced).
- Primary demo path: **MainActivity + activity_main.xml** only.
- Fragment files support a second-week module on Navigation Component.

Document generated for the LayoutEvent Android laboratory module. Align printed copies with the project version in your repository.